

ScholarPath

AI Architecture Deep Dive

Retrieval-Augmented Generation, Rule-Based Scoring & Eligibility Checking — a comprehensive technical reference for the academic defence. Every diagram and code snippet reflects the live production codebase.

Stack: .NET 10 · React 19 · Azure OpenAI (gpt-4o-mini + text-embedding-3-small) · SQL Server · 692 KB documents · Standard RAG (not Agentic).

Contents

1	Core Concepts	2
1.1	Embeddings — Turning Text into Numbers	2
1.2	Cosine Similarity — Measuring Semantic Closeness	2
1.3	What is RAG?	3
2	Standard RAG vs. Agentic RAG	3
2.1	Three Failure Modes of Standard RAG	3
2.2	Agentic RAG — The Control Loop	4
2.3	Decision Framework: Which Approach to Use?	4
3	ScholarPath AI Architecture — The Full Picture	4
4	Knowledge Base — Building and Maintaining It	5
4.1	Five Data Sources	5
4.2	Rebuild Algorithm	6
4.3	Vector Storage — SQL Server, Not a Vector Database	6
5	Chatbot — Full RAG Pipeline (9 Steps)	7
5.1	System Prompt Design	7
5.2	Session History	7
6	Recommendations — Rule-Based Scoring (No LLM)	8
6.1	Scoring Formula	8
6.2	Cache Architecture and Re-Hydration	8
7	Eligibility Check — Profile Matching	9
7.1	Three Criteria	9
7.2	Verdict Derivation	9
8	Fallback Chain — Graceful Degradation	10
9	Cost Model and Fine-Tuning Roadmap	10
9.1	Production Cost (GPT-4o-mini)	10
9.2	Fine-Tuning Roadmap	10
10	Admin Knowledge Base Management	11
10.1	Four Actions on the Admin Page	11
10.2	Incremental vs. Force Rebuild	11

11 Fine-Tuning Pipeline (In-App)	11
11.1 JSONL Dataset Generation	12
11.2 Starting a Fine-Tuning Job from the Admin Panel	12
11.3 Checking Job Status	13
11.4 Activating the Fine-Tuned Model	13
11.5 Admin API Endpoints	13
12 AiInteraction Entity — The Caching Layer	13
12.1 Entity Structure	13
12.2 ResponseText by Feature	14
12.3 Full Recommendation Cache Lifecycle	14
12.4 Related Entities	15
13 Configuration Reference	15
14 Summary	15

1. Core Concepts

1.1 Embeddings — Turning Text into Numbers

Concept: Embedding

An **embedding** converts any text — a word, sentence, or paragraph — into a fixed-length array of floating-point numbers called a *vector*. The fundamental property: **semantically similar texts produce numerically close vectors**, regardless of exact wording or even language. A question in English and the same question in Arabic yield vectors that point in nearly the same direction in a 1,536-dimensional space.

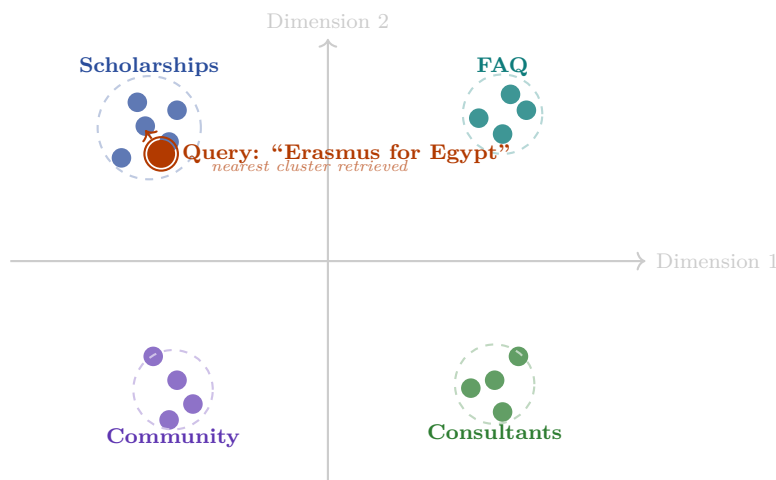


Figure 1. Conceptual 2-D embedding space (real vectors have 1,536 dimensions). The query vector (red circle) lands near the Scholarship cluster. The retriever returns those docs as context for the LLM.

In ScholarPath the embedding model is **Azure OpenAI text-embedding-3-small** producing **1,536-dimensional** vectors. A `sp:` prefix namespaces the model name in the database so vectors from different models are never mixed in search:

Listing 1. AzureOpenAiEmbeddingService.cs — ModelName

```
public string ModelName =>
    IsAzureEmbeddingConfigured()
        ? $"sp:{opts.Value.AzureOpenAi.EmbeddingDeploymentName}"
        // ^ "sp:text-embedding-3-small" -- sp: is a namespace prefix only
        : local.ModelName;    // fallback when Azure is not configured
```

1.2 Cosine Similarity — Measuring Semantic Closeness

Given two embedding vectors **a** and **b**, cosine similarity measures the *angle* between them (not their lengths):

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}} \in [0, 1]$$

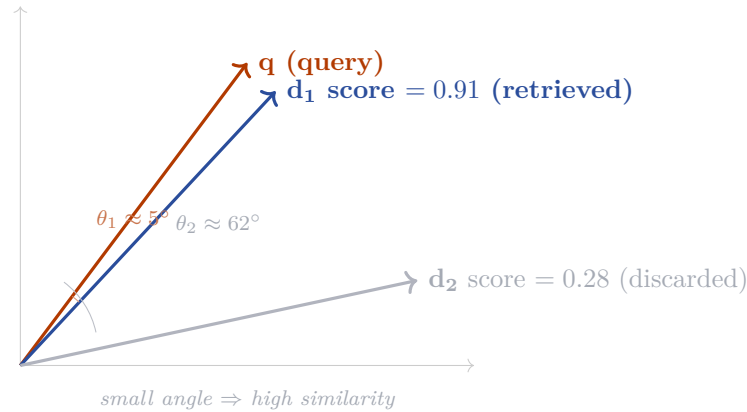


Figure 2. Two documents against a query. $\mathbf{d}_1$ (small angle, score 0.91) lies above the `RagMinScore` = 0.6 threshold and is included in the LLM context block. $\mathbf{d}_2$ is discarded.

Score	Interpretation	Action in ScholarPath
≥ 0.85	Near-identical meaning	Retrieved & cited in response
0.70–0.84	Very similar	Retrieved & cited
0.60–0.69	Clearly related	Retrieved (at <code>RagMinScore</code> threshold)
< 0.60	Weakly related	Discarded before LLM prompt

1.3 What is RAG?

Concept: Retrieval-Augmented Generation (RAG)

RAG solves the *knowledge gap* of LLMs. Instead of answering purely from training memory (which may be stale or hallucinated), the LLM receives **retrieved, grounded context** from a live knowledge base alongside the user’s question. The context anchors the answer to real, up-to-date data.

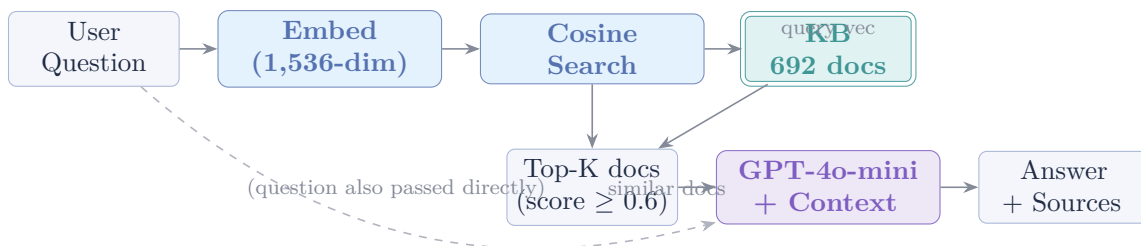


Figure 3. Standard RAG — one linear pass. Embed the question, retrieve relevant documents, inject them as context, generate a grounded answer.

2. Standard RAG vs. Agentic RAG

2.1 Three Failure Modes of Standard RAG

Standard RAG is a *linear* pipeline with no evaluation step. It fails in three patterns identified in the ByteByteGo reference:

1. **Ambiguous queries** — “How do I apply?” lacks context. Is the student asking about the platform signup or a specific scholarship application? Standard RAG retrieves without clarifying.
2. **Scattered evidence** — A question requiring synthesis from multiple document clusters (e.g.

“compare Erasmus to Chevening on funding”) gets a partial answer because the pipeline exhausts a single retrieval pass.

3. **False confidence** — Retrieved documents score high on cosine similarity but do not actually contain the answer. The LLM fills the gap with plausible but invented details.

2.2 Agentic RAG — The Control Loop

Agentic RAG replaces the linear pipeline with a **decision loop** driven by an autonomous agent that can clarify, retry, and route across sources.

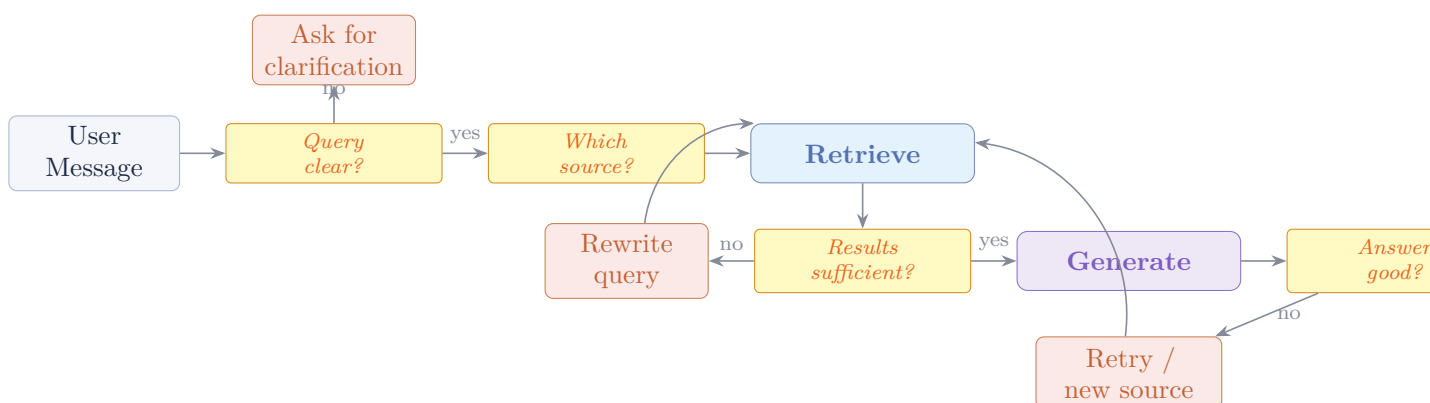


Figure 4. Agentic RAG control loop. The agent (yellow boxes) decides which source to query, evaluates whether results are adequate, and retries with a refined query if not.

2.3 Decision Framework: Which Approach to Use?

Criterion	Standard RAG	Agentic RAG
Response latency	1–3 s	5–15+ s (multiple LLM calls)
Cost per query	≈ \$0.0002	≈ \$0.001–\$0.002 (3–10×)
Simple, single-source Q	Excellent	No improvement
Complex, multi-source Q	Moderate	Significantly better
Debugging	Deterministic, testable	Non-deterministic, hard to reproduce
Optimal corpus size	Up to ~10K docs	10K+ docs, multi-domain

Why this design?

ScholarPath uses **Standard RAG** for the chatbot. The corpus has 692 documents and queries are domain-specific (scholarships, consultants, application tips). A single retrieval pass consistently finds the right document in <10 ms. Adding an agentic loop would multiply latency by 5× and cost by 10× with no measurable accuracy gain on this query set. The threshold for migrating to Agentic RAG is approximately 10,000+ documents with heterogeneous sub-domains.

3. ScholarPath AI Architecture — The Full Picture

Key Point: Critical distinction

ScholarPath has **three AI features**, but only one uses real RAG + GPT. The other two use a deterministic, zero-network scoring algorithm.

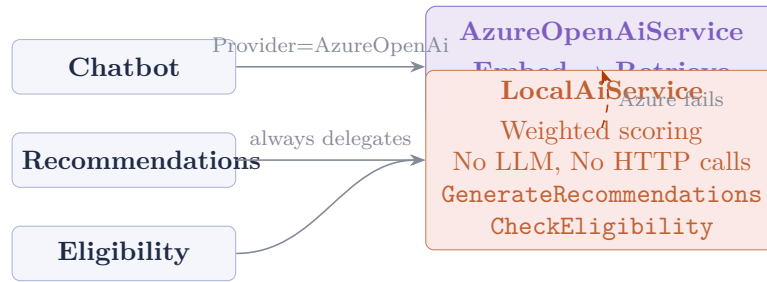


Figure 5. ScholarPath AI routing. The chatbot goes through AzureOpenAiService (real RAG + GPT). Recommendations and eligibility always use LocalAiService regardless of the `Ai:Provider` setting.

Feature	Uses GPT?	Uses RAG?	Approach
Chatbot	✓ Azure GPT-4o-mini	✓ Cosine search	Standard RAG
Recommendations	×	×	Weighted scoring (local)
Eligibility	×	×	Profile matching (local)

Listing 2. AzureOpenAiService.cs — delegation

```

// Recommendations and eligibility use deterministic scoring, no GPT needed.
public Task<AiRecommendationResult> GenerateRecommendationsAsync(...)
    => local.GenerateRecommendationsAsync(...); // always LocalAiService

public Task<AiEligibilityResult> CheckEligibilityAsync(...)
    => local.CheckEligibilityAsync(...); // always LocalAiService

// Only AskAsync uses real Azure RAG + GPT.
public async Task<AiChatResponse> AskAsync(...) { /* RAG pipeline */ }

```

4. Knowledge Base — Building and Maintaining It

4.1 Five Data Sources

The `KnowledgeBaseIndexer.RebuildAsync()` aggregates data from **five distinct sources**, each projecting records into a `KnowledgeDocument` row in SQL Server:

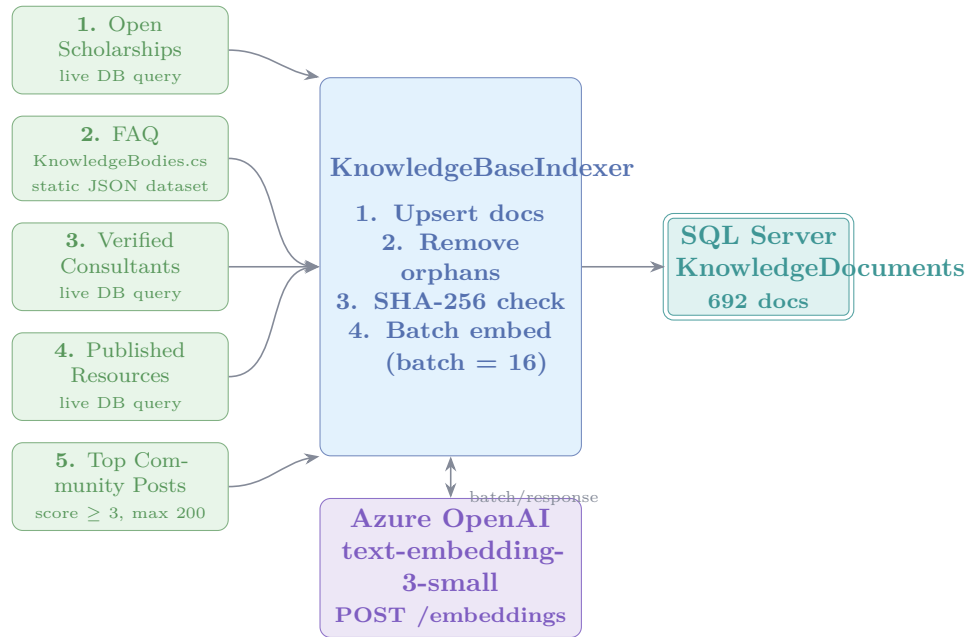


Figure 6. Knowledge base indexer aggregates 5 live data sources. Only documents whose content hash has changed (SHA-256 on `ContentEn` + `ContentAr`) are re-embedded, making incremental rebuilds cheap. Double border = database.

4.2 Rebuild Algorithm

- Upsert** — For each desired document, compute $\text{SHA-256}(\text{ContentEn} + \text{ContentAr})$. If the hash changed, clear the embedding. If new, insert.
- Orphan removal** — Documents in the DB whose source entity no longer exists are deleted.
- Selective re-embedding** — Only documents with an empty embedding, a model mismatch, or `force=true` are sent to Azure. On a warm base this is zero work.
- Batch embedding** — Sent in batches of 16 to `/openai/deployments/text-embedding-3-small/embeddings`. The returned `float[1536]` is packed into `byte[6144]` via `Buffer.BlockCopy` and stored in a `VARBINARY` column.

4.3 Vector Storage — SQL Server, Not a Vector Database

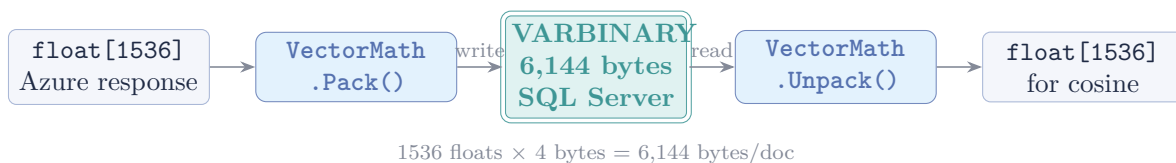


Figure 7. Zero-copy vector serialisation via `Buffer.BlockCopy`. SQL Server is sufficient at 692 documents — full in-memory cosine scan completes in < 10 ms.

Why this design?

A dedicated vector database (Pinecone, Qdrant, pgvector) would add an external deployment, an additional network hop, and operational cost. At 692 documents the full in-memory scan is faster than any remote call. The migration threshold to a vector DB is approximately 10,000–50,000 documents, where approximate nearest-neighbour (ANN) indexing becomes necessary.

5. Chatbot — Full RAG Pipeline (9 Steps)

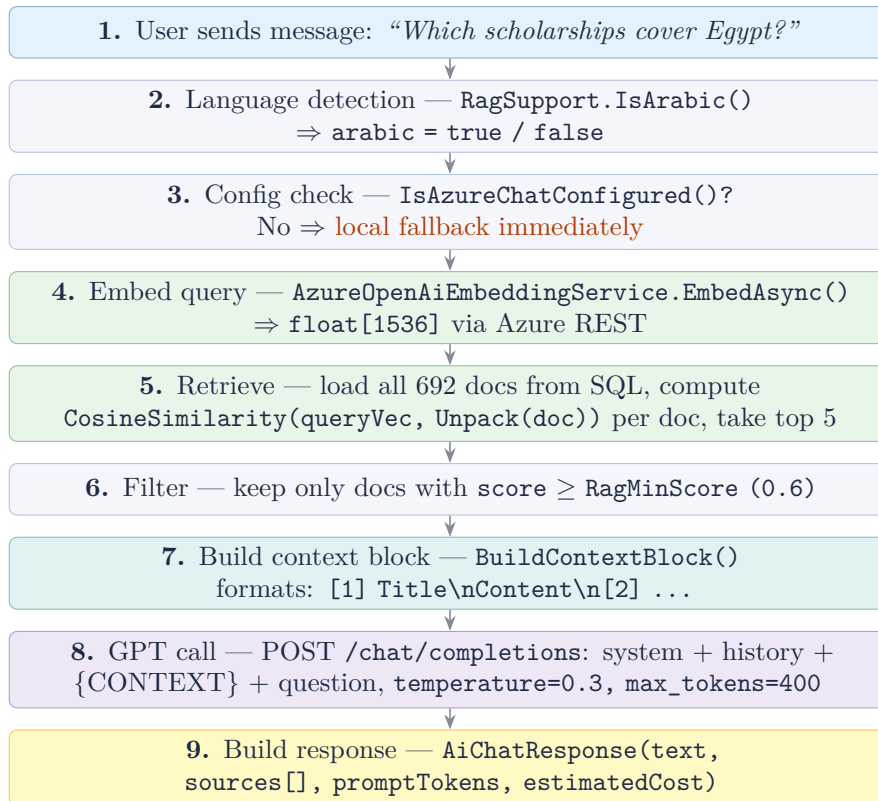


Figure 8. Complete chatbot pipeline. Green steps invoke Azure Embeddings API. Purple step invokes GPT-4o-mini. Total end-to-end latency: $\approx 1\text{--}3\text{s}$.

5.1 System Prompt Design

The system prompt (English and Arabic variants) explicitly instructs the LLM to:

- Cite scholarships, consultants, and resources by name when present in the CONTEXT block
- Draft SoPs, recommendation letters, CVs, and motivation letters on request
- Coach on IELTS/TOEFL/GRE and interview preparation
- Explain eligibility requirements and funding mechanics
- **Never** invent scholarship names, deadlines, or amounts not in CONTEXT
- **Never** refuse a reasonable user request

5.2 Session History

Every API call prepends prior conversation turns between the system prompt and the current question, giving the model session memory without any server-side store:

Listing 3. ChatCompletionAsync — session history injection

```

var messages = new List<object>
{
    new { role = "system", content = system },
};
foreach (var turn in history) // prior turns from this session
    messages.Add(new { role = turn.Role, content = turn.Content });
messages.Add(new { role = "user", content = user }); // current question
  
```


6. Recommendations — Rule-Based Scoring (No LLM)

Important Note

There is **no GPT call**, **no RAG**, **no HTTP request** in the recommendation pipeline. It is a fully deterministic weighted-scoring algorithm that runs entirely in-process on the application server.

6.1 Scoring Formula

$$\text{Score} = \underbrace{S_{\text{level}}}_{\leq 30} + \underbrace{S_{\text{field}}}_{\leq 40} + \underbrace{S_{\text{country}}}_{\leq 25} + \underbrace{B_{\text{funding}}}_{\leq 5} \in [0, 100]$$

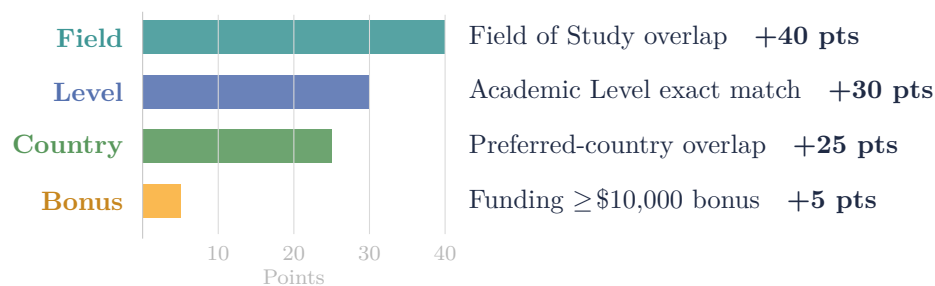


Figure 9. Recommendation score component weights. Field of study is the largest factor (40 pts) so subject alignment dominates over geography.

Score	Label	Explanation shown to student
80–100	Strong match	“Strong match — aligns with your preferred fields and level.”
50–79	Partial match	“Partial match — overlaps with some of your preferences.”
20–49	Loose match	“Loose match — touches areas you’ve listed; worth a look.”
0–19	Weak match	“Weak match — doesn’t align well with your current profile.”

6.2 Cache Architecture and Re-Hydration

Scores are computed once and cached as JSON in `AiInteraction.ResponseText`. The cached shape (`RecommendationItemDto`) stores only AI-generated fields. Live deadline and funding data are **always fetched fresh from the DB** so a date change on a scholarship is reflected immediately:

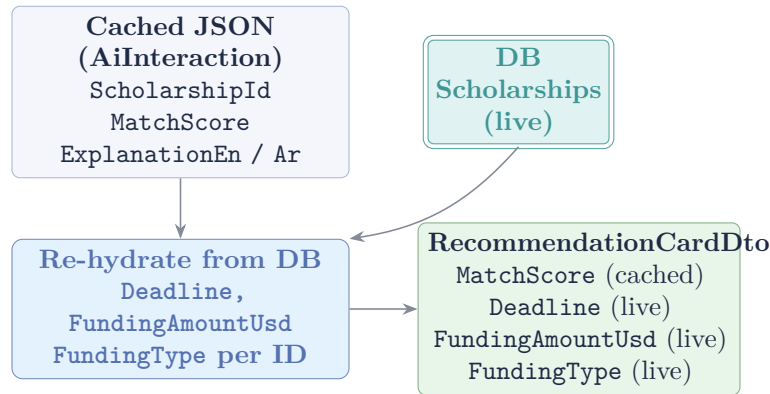


Figure 10. Cache re-hydration. The stored JSON contains only AI-computed fields. Deadline and funding are always fetched live so stale dates never appear in the UI.

7. Eligibility Check — Profile Matching

7.1 Three Criteria

Criterion	Student source	Scholarship source	Match logic
Academic level	profile.AcademicLevel	TargetLevel	Exact equality
Country	PreferredCountries + nationality	TargetCountriesJson	CountryNormalizer
Field of study	profile.FieldOfStudy	FieldsOfStudyJson	Case-insensitive contains

Per-criterion match values: "yes" / "partial" / "no" / "unknown" (when the student's profile field is empty).

7.2 Verdict Derivation

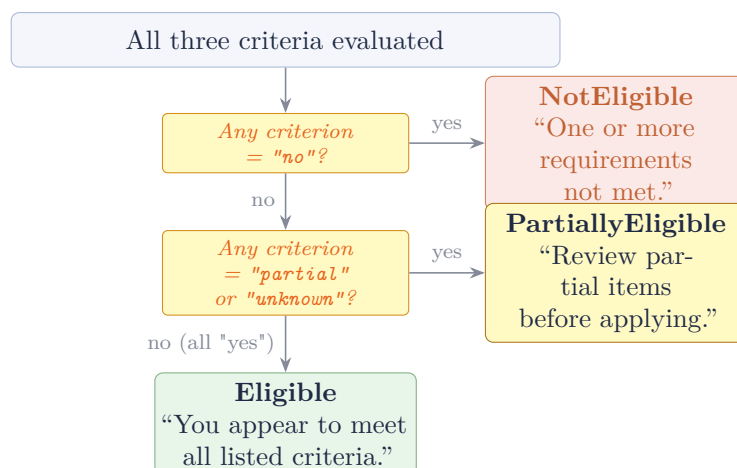


Figure 11. Verdict derivation in `LocalAiService.DeriveVerdict()`. A single "no" is sufficient for `NotEligible`; any uncertainty yields `PartiallyEligible`.

Note

Country normalisation is non-trivial. A scholarship listing "France" must match a student who typed "FR", "france", or "France". `CountryNormalizer.Matches()` collapses all representations to a canonical key before comparison. Without this, an Egyptian student preferring Germany

would read “no” on a scholarship listing the country as “DE”.

8. Fallback Chain — Graceful Degradation

Every AI call is wrapped in a multi-level fallback that ensures the user always receives a response, even if Azure is completely unreachable.

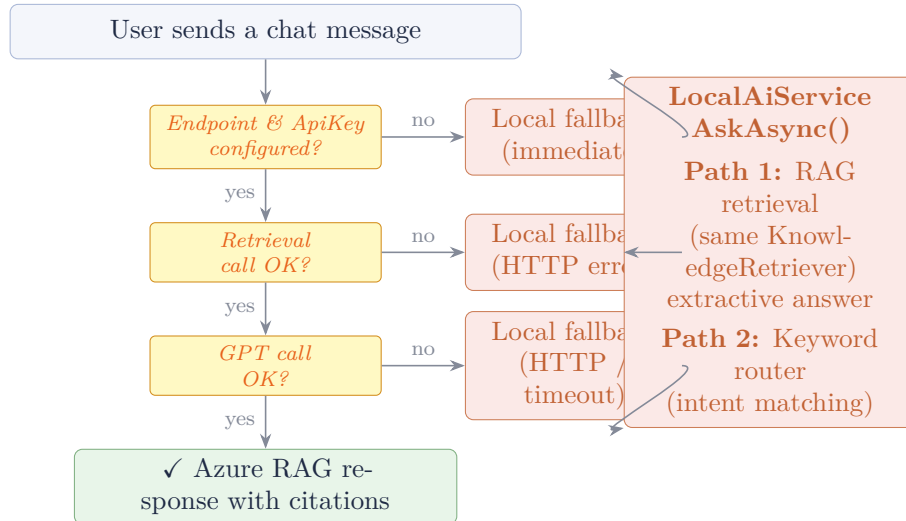


Figure 12. Three-level fallback chain for the chatbot. Any failure routes to `LocalAiService.AskAsync()`, which runs the same RAG retrieval without GPT, then falls back to a keyword router.

9. Cost Model and Fine-Tuning Roadmap

9.1 Production Cost (GPT-4o-mini)

$$\text{Cost} = N_{\text{prompt}} \times \frac{\$0.15}{10^6} + N_{\text{completion}} \times \frac{\$0.60}{10^6}$$

Scenario	Prompt tok.	Completion tok.	Cost
Simple chat turn	300	150	≈ \$0.000135
Rich context (5 docs retrieved)	600	300	≈ \$0.000270
1,000 turns/day	—	—	≈ \$0.27
DailyCostLimitUsd cap	\$5.00 — hard limit in <code>AiOptions</code>		

9.2 Fine-Tuning Roadmap

A fine-tuning dataset (`scholarpath-finetune.jsonl`, 332 training examples) has been generated in OpenAI chat format. The codebase already contains the selection logic — no code change is required to activate it:

Listing 4. Automatic fine-tuned model selection

```

var deployment = string.IsNullOrEmpty(az.FineTunedDeploymentName)
    ? az.DeploymentName // "gpt-4o-mini" (current)
    : az.FineTunedDeploymentName; // e.g. "scholarpath-ft-001"
  
```

Setting the App Service environment variable `Ai__AzureOpenAi__FineTunedDeploymentName` is sufficient to switch models.

10. Admin Knowledge Base Management

10.1 Four Actions on the Admin Page

The page `AdminKnowledgeBase.tsx` exposes four operations on the knowledge base, each wired to a dedicated API endpoint:

Button		API call	Effect
Rebuild (incremental)		POST <code>.../knowledge-base/rebuild?force=false</code>	Re-embeds only changed docs
Rebuild (force)		POST <code>.../knowledge-base/rebuild?force=true</code>	Re-embeds ALL 692 docs
Import + Rebuild		POST <code>.../datasets/import</code>	Fetches external scholarships, then rebuilds
Export Fine-Tuning		GET <code>.../fine-tuning/dataset</code>	Downloads <code>scholarpath-finetune.jsonl</code>

The rebuild response is a `KnowledgeBaseRebuildResultDto`:

Listing 5. Admin UI receives this on every rebuild

```
{
  "upserted": 12,    // new or content-changed docs saved to DB
  "reembedded": 3,  // docs whose embedding was refreshed (hash changed)
  "removed": 2      // orphan docs deleted (source entity no longer exists)
}
```

10.2 Incremental vs. Force Rebuild

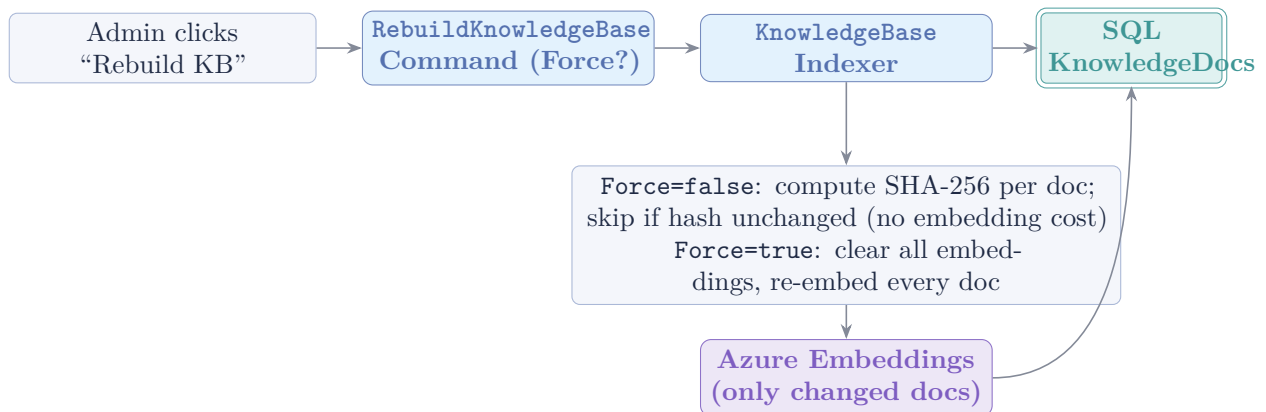


Figure 13. Rebuild decision flow. `Force=false` is the default — only documents whose `SHA-256(ContentEntity + ContentArticle)` changed are sent to Azure. A warm rebuild typically costs \$0.00 in embedding tokens.

Why this design?

The SHA-256 check makes nightly Hangfire-triggered rebuilds essentially free. Changing one scholarship’s description re-embeds exactly one document ($1,536 \text{ floats} = \$0.000002$). A force rebuild re-embeds all 692 documents (cost: $692 \times 1536 \times \$0.02/10^6 \approx \0.021).

11. Fine-Tuning Pipeline (In-App)

Fine-tuning in ScholarPath is fully managed from the admin panel — no PowerShell scripts, no manual file uploads. The pipeline covers four steps: export, upload, train, and activate.

11.1 JSONL Dataset Generation

The **Export Fine-Tuning** button downloads `scholarpath-finetune.jsonl` — a supervised fine-tuning dataset in OpenAI chat-completion format. It is generated on-demand by `ExportFineTuningDatasetQuery` with no external calls: all data comes from the live database and the static `KnowledgeBodies.cs` constant.

Each line is one self-contained training example:

Listing 6. Single JSONL training example (truncated)

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are ScholarPath AI, a bilingual assistant..."
    },
    {
      "role": "user",
      "content": "Tell me about the Erasmus+ scholarship."
    },
    {
      "role": "assistant",
      "content": "The Erasmus+ scholarship is a European Union programme
                  that supports student mobility..."
    }
  ]
}
```

Source	Example type	Count (approx.)
KnowledgeBodies.cs	FAQ English Q&A pairs	~32
KnowledgeBodies.cs	FAQ Arabic Q&A pairs	~32
Open Scholarships (DB)	Overview in English	≤100
Open Scholarships (DB)	Overview in Arabic	≤100
Open Scholarships (DB)	Eligibility requirements	≤68
Total		~332

11.2 Starting a Fine-Tuning Job from the Admin Panel

The admin clicks **Start fine-tuning job** on the Knowledge Base page. Under the hood `StartFineTuningJobCommand` performs three steps in one HTTP call:

1. Calls `ExportFineTuningDatasetQuery` internally to obtain the JSONL.
2. Uploads the file to the Azure OpenAI Files API (`POST /openai/files`) via `AzureFineTuningService`, then polls `GET /openai/files/{id}` until `status = "processed"`.
3. Creates the fine-tuning job: `POST /openai/fine_tuning/jobs` with `model = "gpt-4o-mini"` and the uploaded `training_file` ID.

The resulting `jobId` and initial status `"submitted"` are persisted in the `PlatformSettings` table under the keys shown below. The job runs asynchronously on Azure (typically 30–90 minutes).

Listing 7. PlatformSettings keys for fine-tuning state

```
ai.fineTuning.lastJobId      -- Azure job ID (e.g. "ftjob-abc123")
ai.fineTuning.lastJobStatus  -- last-known status string
ai.fineTuning.lastFinishedModel -- model name once the job succeeds
ai.fineTuning.activeDeploymentName -- currently active fine-tuned deployment
```

11.3 Checking Job Status

The admin clicks **Check status** (or the client polls automatically). `GetFineTuningJobStatusQuery` reads the stored `jobId` from `PlatformSettings`, calls `GET /openai/fine_tuning/jobs/{id}` on Azure, writes the fresh status back to `PlatformSettings`, and returns:

Listing 8. `FineTuningStatusDto` returned by `GET /api/admin/ai/fine-tuning/status`

```
public sealed record FineTuningStatusDto(
    string? JobId,
    string Status, // "submitted" | "running" | "succeeded" | "failed"
    string? FineTunedModel, // model name once succeeded
    string? Error,
    string? ActiveDeploymentName,
    bool HasActiveDeployment);
```

11.4 Activating the Fine-Tuned Model

Once the job status is "succeeded" and the admin has created a deployment in Azure AI Foundry from the finished model, they enter the deployment name in the admin panel and click **Activate**.

`ActivateFineTunedModelCommand` writes the deployment name to `PlatformSettings("ai.fineTuning.activeDeploymentName")`. No App Service restart is required.

`AzureOpenAiService` resolves the deployment on every chat call using this three-tier priority:

Listing 9. Deployment selection in `AzureOpenAiService.cs`

```
// Priority: DB platform setting -> appsettings override -> base model
var dbDeployment = await PlatformSettingsReader.GetStringAsync(
    db, PlatformSettingsKeys.ActiveFineTunedDeploymentName, null, ct);
var deployment = !string.IsNullOrEmpty(dbDeployment)
    ? dbDeployment
    : !string.IsNullOrEmpty(az.FineTunedDeploymentName)
    ? az.FineTunedDeploymentName
    : az.DeploymentName;
```

The admin can also click **Revert to base model** to clear the platform-setting and restore `gpt-4o-mini` immediately.

11.5 Admin API Endpoints

Method	Route	Purpose
GET	<code>/api/admin/ai/fine-tuning/dataset</code>	Export JSONL (download)
POST	<code>/api/admin/ai/fine-tuning/start</code>	Upload + start job
GET	<code>/api/admin/ai/fine-tuning/status</code>	Poll job status
PUT	<code>/api/admin/ai/fine-tuning/activate</code>	Set active deployment
DELETE	<code>/api/admin/ai/fine-tuning/activate</code>	Revert to base model

12. AiInteraction Entity — The Caching Layer

12.1 Entity Structure

`AiInteraction` is the **single table for all AI operations** — it serves as both an audit log and a cache.

Listing 10. `Domain/Entities/AI.cs` (condensed)

```
public class AiInteraction : AuditableEntity
{
    public Guid UserId { get; set; }
```

```

public AiFeature Feature          { get; set; } // Recommendation / Chat / Eligibility
public AiProvider Provider        { get; set; } // AzureOpenAi / Local
public string? SessionId          { get; set; } // groups chat turns into a session

// The payload -- meaning varies by Feature:
public string PromptText          { get; set; } // what was asked
public string ResponseText        { get; set; } // answer text OR cached JSON
public string? MetadataJson       { get; set; } // structured breakdown (eligibility)

// Billing
public int PromptTokens           { get; set; }
public int CompletionTokens       { get; set; }
public decimal CostUsd            { get; set; }

public DateTimeOffset StartedAt    { get; set; }
public DateTimeOffset? CompletedAt { get; set; }
public string? ErrorMessage        { get; set; }
}

```

12.2 ResponseText by Feature

Feature	ResponseText content	Read by
Recommendation	List<RecommendationItemDto> as JSON	GetMyRecommendationsQuery
Chat	Plain assistant reply text	GetAiSessionTurnsQuery
Eligibility	Verdict string; detail in MetadataJson	Client displays directly

12.3 Full Recommendation Cache Lifecycle

WRITE (generate)

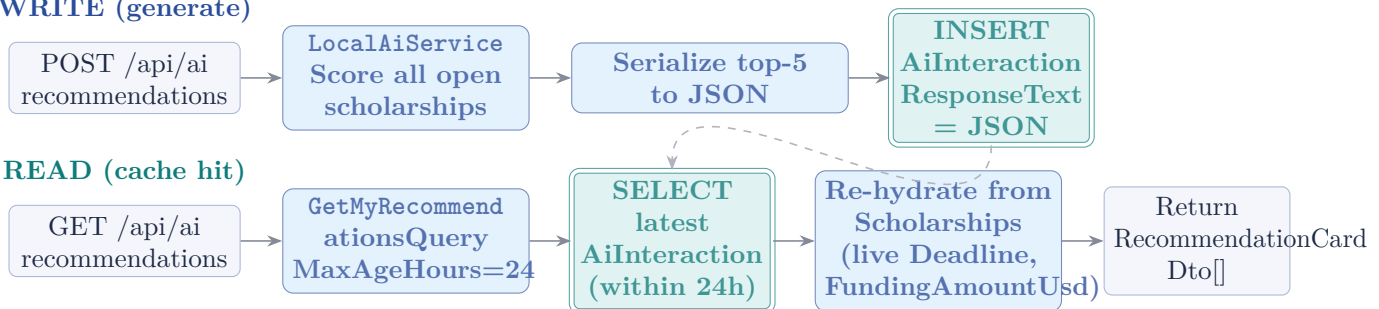


Figure 14. Recommendation cache lifecycle. The write path (generate) runs once and stores scored items as JSON. The read path (cache hit) deserialises the JSON and re-hydrates live deadline/funding data so stale values never appear in the UI. `MaxAgeHours=24` — cached results older than 24 h are treated as missing and trigger a fresh generation.

12.4 Related Entities

Entity	Purpose
RecommendationClickEvent	Logged when user clicks a recommendation card. Tracks Source (“card”, “email”, etc.) for CTR analysis.
AiRedactionAuditSample	Monthly PII-redaction compliance sample — stores a random chat turn and verifies no personal data leaked into responses.

13. Configuration Reference

Setting key	Production value	Effect
Ai:Provider	AzureOpenAi	Route chatbot through Azure
Ai:RagTopK	5	Maximum documents retrieved per query
Ai:RagMinScore	0.6	Minimum cosine similarity to include
Ai:DailyCostLimitUsd	5.0	Hard daily spend cap
AzureOpenAi:DeploymentName	gpt-4o-mini	Chat generation model
AzureOpenAi:EmbeddingDeploymentName	text-embedding-3-small	Embedding model
AzureOpenAi:EmbeddingDimensions	1536	Vector size
AzureOpenAi:FineTunedDeploymentName	(empty)	Post-launch fine-tuned model

14. Summary

Key Point: One-sentence summary per feature

Chatbot: user message → Azure Embed → cosine search (692 docs, in-memory) → top-5 docs filtered at 0.6 → GPT-4o-mini with context → cited answer.

Recommendations: student profile → weighted score per open scholarship (level 30 + field 40 + country 25 + bonus 5) → top-5 ranked list; cached as JSON, deadline/funding re-hydrated from DB on every read.

Eligibility: student profile vs. scholarship criteria → yes/partial/no per criterion → **Eligible** / **PartiallyEligible** / **NotEligible** verdict.

Feature	Uses GPT?	Uses RAG?	Approach
Chatbot	✓ Azure GPT-4o-mini	✓ Cosine search	Standard RAG
Recommendations	×	×	Weighted scoring (deterministic)
Eligibility	×	×	Profile matching (deterministic)

Primary reference. ByteByteGo, *How Agentic RAG Works*, <https://blog.bytebytego.com/p/how-agentic-rag-works>
Implementation. `server/src/ScholarPath.Infrastructure.Services/` (`AzureOpenAiService.cs`, `KnowledgeBaseIncubator.cs`, `KnowledgeRetriever.cs`, `LocalAiService.cs`, `VectorMath.cs`, `RagSupport.cs`).